Лекция 3
Последовательности и потоки



01

Последовательности и потоки

Как вычислять данные по требованию и строить потоковые pipeline?

Коллекция и последовательность

Коллекция — структура данных.

Последовательность — абстракция последовательного доступа к элементам.

Одинаковые операции, разные источники

```
(first [1 2 3])
(rest [1 2 3])

(first '(1 2 3))
(rest '(1 2 3))
```

vector / list → sequence

Отложенные вычисления

```
(def x
  (delay
    (println "computing...")
    42)) ;; вычислений нет

(force x)
;; печатает "computing..."
;; => 42

(force x)
;; повторно не вычисляется
;; => 42
```

Когда вычисляется map?

```
(def xs
  (map #(* % %) [1 2 3 4]))
```

при def? при first? при take?

Lazy sequence: вычисление по требованию

```
(defn compute [x]
  (println "compute" x)
  (* x x)) ;; определяем функцию

(def xs
  (map compute '(1 2 3 4))) ;; ничего не печатается

(first xs) ;; печатает: compute 1

(first xs) ;; ничего не печатается
```

Реализация lazy sequence

Реализация (realization) — вычисление отложенной части ленивой последовательности в ответ на запрос ее значения.

После реализации полученный результат сохраняется и повторно не вычисляется.

Реализуется не вся последовательность, а только затребованная её часть

Реализация map

```
(def xs (map compute '(1 2 3 4)))

(type xs) ;; => clojure.lang.LazySeq

(realized? xs) ;; => false

(first xs) ;; требуем первый элемент

(realized? xs) ;; => true
(realized? (rest xs)) ;; => false
```

Lazy sequence и chunking

```
(def xs
  (map compute [1 2 3 4])) ;; ничего не печатается

(first xs)
;; печатает:
;; compute 1
;; compute 2
;; compute 3
;; compute 4
;; => 1
```

Композиция lazy sequence

```
(def xs
  (take 3
    (filter even?
      (map compute '(1 2 3 4 5 6 7)))))
;; пока ничего не вычислено

(first xs)
;; => 4
```

Часто используемые lazy sequence-функции

Источники

```
(range ...)   (iterate f x)   (repeat x)
(repeatedly f)   (cycle coll)
```

Преобразования

```
(map f xs)   (filter p xs)   (keep f xs)
(mapcat f xs)
```

Ограничение / пропуск

```
(take n xs)   (drop n xs)
(take-while p xs)   (drop-while p xs)
```

lazy-seq

```
(defn naturals [n]
  (lazy-seq
    (cons n (naturals (inc n)))))

(println (take 5 (naturals 1)))
;; => (1 2 3 4 5)
```

lazy-seq: пошаговая реализация

```
(defn naturals [n]
  (lazy-seq
    (println "realize body:" n)

    (let [xs (cons n (naturals (inc n)))]
      (println "head:" (first xs))
      (println "tail type:" (type (rest xs)))
      (println "tail realized?:" (realized? (rest xs)))
      xs)))
```

Трассировка реализации

```
(def xs (naturals 1)) ;; ничего не печатается
(first xs)
;; realize body: 1
;; head: 1
;; tail type: clojure.lang.LazySeq
;; tail realized?: false
;; => 1
(second xs)
;; realize body: 2
;; head: 2
;; tail type: clojure.lang.LazySeq
;; tail realized?: false
;; => 2
```

Что находится в памяти?

После `(def xs (naturals 1))`

```
xs ──▶ LazySeq₁ [не реализован]
```

После `(first xs)`

```
xs ──▶ LazySeq₁ [реализован]
                  │
                  ▼
                Cons
                ├─ first: 1
                └─ rest ──▶ LazySeq₂ [не реализован]
```

iterate

```
(def naturals
  (iterate inc 1))

(take 5 naturals)
;; => (1 2 3 4 5)
```

iterate строит бесконечную lazy sequence:
1, (inc 1), (inc 2), ...

Пример: последовательность Фибоначчи

```
(def fib-pairs
  (iterate (fn [[a b]] [b (+ a b)]) ;;destructuring
           [0 1]))
;; состояние на каждом шаге — пара [a b]:
[a b] → [b (+ a b)]

(def fib-seq (map first fib-pairs))

(take 5 fib-seq) ;; => (0 1 1 2 3)
```

Бесконечные последовательности

```
(range)          => 0, 1, 2, ...
(iterate inc 0) => 0, 1, 2, ...
(repeat 1)       => 1, 1, 1, ...
(cycle [1 2 3]) => 1, 2, 3, 1, 2, ...
```

любой конечный префикс можно вычислить за конечное время

Что произойдет?

```
(count (range))
```

вернёт число?
зависнет?
ошибка?

Pipeline

```
source  →  stage 1  →  stage 2  →  result
```

Пример Bash:

```
cat access.log | grep ERROR | sort | uniq -c
```

Pipeline — композиция этапов обработки, где результат одного этапа становится входом следующего.

Пример: пайплайн обработки событий

```
(defn generate-event []
  {:request-id (random-uuid)
   :type       (rand-nth [:GET :POST])
   :time-ms    (rand-int 1000)})

(defn get-type [event]
  (:type event))

(defn get-time [event]
  (:time-ms event))
```

Пример: пайплайн обработки событий

```
(def events (repeatedly 100 generate-event))
;; источник данных
;; {:request-id
"48f79e31-6001-4edc-b810-3405958a96bd",
:type :GET, :time-ms 216}

(->> events
     (filter #(= :GET (get-type %)))
     (map get-time)
     (reduce max)) ;; максимальное время ответа
```

Модель:
Producer → transformations → consumer

```
events → filter :GET → map :time-ms → reduce max
```

- **producer**: генерация событий
- **transformations**: filter, map
- **consumer**: reduce max

reduce запрашивает элементы и тем самым
реализует lazy sequence.

Pull модель

Поток данных:
producer → filter → map → consumer

Запросы результата:
producer ← filter ← map ← consumer

В lazy pipeline вычисление инициирует consumer: запрос результата распространяется upstream к producer.

Что даёт lazy pipeline?

`source → map → filter → take → consumer`

Промежуточные результаты не обязаны материализовываться целиком.

Элементы проходят через pipeline по мере запроса downstream.

От lazy sequence к stream

`lazy sequence` → данные вычисляются постепенно

`stream` → данные поступают постепенно во времени

Сходство: не нужен весь набор данных заранее.

Различие: lazy sequence — локальная модель вычисления. Stream processing добавляет время, асинхронность, состояние и управление потоком.

Batch vs Stream

Batch processing:
- данные группируются в конечные порции;
- обработка запускается для каждой порции как отдельная работа;
- результат появляется после обработки батча;
- источник может быть непрерывным.

Stream processing:
- элементы обрабатываются по мере поступления;
- нет необходимости ждать формирования полного батча;
- вычисление обычно долгоживущее;
- результаты могут обновляться непрерывно.

Границы композиции lazy sequence

Ленивость убирает полную материализацию, но не все накладные расходы.

map → filter → take
→ каждый stage создает новую sequence-обертку

(map f xs), (filter p xs)
→ преобразование - “зашито” в результат, трудно переиспользовать

Хотим: преобразование отдельно от source и consumer.

02

Transducers

Как отделить описание преобразования от источника данных и способа потребления?

Зачем ещё одна абстракция?

```
(->> xs
     (map f)
     (filter p)
     (take 10))
```

Можно ли описать transformation независимо от source?

От последовательности к преобразованию

```
(map inc '(0 1 2 3))                ;; => seq (1 2 3 4)

(def map-inc (map inc))             ;; => описывает
преобразование
```

transducer

У ряда sequence-функций арность 1 имеет специальный смысл: функция не обрабатывает коллекцию, а возвращает специальную функцию - transducer.

```
(map f)        ;; => transducer
(filter p)     ;; => transducer
(take n)       ;; => transducer
(drop n)       ;; => transducer
```

Композиция преобразований

```
(def xf
  (comp
    (filter #(= :GET (get-type %)))
    (map get-time)))

;; xf пока ничего не обрабатывает
;; это описание pipeline преобразований

event → filter :GET → map get-time
```

Где использовать transducer?

```
(transduce xf rf init source)
;; редукция → итоговое значение

(sequence xf source)
;; lazy sequence без цепочки промежуточных sequence

(into target xf source)
;; применить pipeline и материализовать результат

(eduction xf source)
;; reducible/iterable view: вычисление при
потреблении
```

transduce: получить итоговое значение

```
(def xf
  (comp
    (filter even?)
    (map #(* % %))))

(transduce xf + 0 [1 2 3 4 5])
;; применяем преобразование: фильтрация + возведение
в квадрат
;; затем суммирование при помощи reduce в одно число
;; => 20

;; pipeline исполняется внутри reduce;
;; промежуточные sequence не создаются
```

Почему нет промежуточных sequence?

`input element → filter → map → reducing function`

Один элемент проходит через композицию преобразований непосредственно в reducer.

Между этапами не требуется создавать промежуточные sequence.

into: сразу собрать коллекцию

```
(def xf
  (comp
    (filter even?)
    (map #(* % %))))

(into [] xf [1 2 3 4 5]) ;; преобразуем в vector
;; => [4 16]

(into #{} xf xs)          ;; преобразуем в set
;; => #{4 16}

;; один и тот же xf, target определяет
;; материализованный результат
```

sequence: получить lazy sequence

```
(def xf
  (comp
    (filter even?)
    (map #(* % %))))

(def result (sequence xf [1 2 3 4 5])) ;; ленивая

(first result) ;; => 4
(rest result)  ;; => (16)

;; одна lazy sequence вместо цепочки
;; map-seq → filter-seq → ...
```

eduction: reducible view

```
(def xf
  (comp
    (filter even?)
    (map #(* % %))))

(def view (eduction xf [1 2 3 4 5]))
;; создание view ничего не вычисляет

(reduce + 0 view) ;; => 20
(into [] view)    ;; => [4 16]

;; transformation выполняется consumer'ом
```

sequence vs eduction

`sequence: (sequence xf source)`

- возвращает lazy sequence
- реализованные элементы кэшируются
- повторный доступ не запускает xf снова

`eduction: (eduction xf source)`

- возвращает reducible / iterable view
- преобразованные результаты не кэшируются
- новый обход снова выполняет xf

Один transducer — разные контексты

```
(def xf
  (comp
    (filter even?)
    (map #(* % %))))

(transduce xf + 0 xs)
(into [] xf xs)
(sequence xf xs)
```

Одно преобразование — разные способы исполнения

03

Transducers изнутри

Reducing function: контракт

Reducing function — функция, которую процесс reduction вызывает на разных стадиях.

В transducer protocol она поддерживает три arity:

```
(rf)                    ;; init
→ initial accumulator

(rf acc input)          ;; step
→ new accumulator

(rf acc)                ;; completion
→ final result
```

Пример reducing function

```
(def collect-rf
  (fn
    ([] [])                                  ;; init
    ([acc] acc)                              ;; completion
    ([acc input] (conj acc input))))         ;; step

(collect-rf)                                 ;; => []

(collect-rf [1 2] 3)                         ;; => [1 2 3]

(collect-rf [1 2 3])                         ;; => [1 2 3]
```

Одна функция — три роли в процессе reduction.

Форма transducer

```
(fn [rf]                          ;; принимает reducing function
  (fn                             ;; возвращает новую
    ([] (rf))                        ;; новый init
    ([result] (rf result))           ;; новый completion
    ([result input] ...)))           ;; новый step
```

Init: вызывает (rf)

Step: вызывает step у rf 0 или более раз:
  filter → 0/1, map → 1, cat → много

Completion: вызывает (rf result) ровно один раз

Пример: (map inc) как transducer

```
;; (map inc) возвращает transducer такого вида:
(def map-inc
  (fn [rf]
    (fn
      ([] (rf))
      ([result] (rf result))
      ([result input]
       (rf result (inc input))))))
```

Пример: (map inc) как transducer

```
;; используем rf с прошлого слайда
(def rf' (map-inc collect-rf))

(rf')                    ;; => []
(rf' [1 2] 3)            ;; => [1 2 4]
(rf' [1 2 4])            ;; => [1 2 4]
```

Меняется только step:
input → inc → collect-rf
init и completion просто делегируются в rf.

Как transduce собирает всё вместе

```
(def xf map-inc)      ;; transducer
(def rf collect-rf)   ;; reducing function
(def rf' (xf rf))     ;; новая reducing function

(transduce xf rf [] [1 2 3]) ;; => [2 3 4]

;; внутри:        (rf' acc   value)
[]        + 1 => (rf' []     1   )       → [2]
[2]       + 2 => (rf' [2]    2   )       → [2 3]
[2 3]     + 3 => (rf' [2 3]  3   )       → [2 3 4]

completion: (rf' [2 3 4]) => [2 3 4]
```

Stateful transducers

```
(take 10)

;; преобразование может хранить состояние
;; между входными элементами

(take 10) должен помнить,
сколько элементов уже прошло.
```

Ранний возврат: reduced

```
(defn sum-until-4 [acc x]
  (if (= x 4)
    (reduced acc)    ;; вызов reduced оборачивает acc
    (+ acc x)))

(reduce sum-until-4 0 [1 2 3 4 5 6])
;; => 6

acc=0, x=1 → 1
acc=1, x=2 → 3
acc=3, x=3 → 6
acc=6, x=4 → (reduced 6) → STOP
```

take и ранний возврат

```
(take 10)

;; transducer take использует тот же механизм
reduced, чтобы после N элементов остановить
reduction.
```

Lazy sequence vs Transducer

Lazy sequence

- представляет sequence
- связана с sequence abstraction
- лениво реализует элементы
- (map f xs): sequence

Transducer

- представляет transformation
- не зависит от конкретного source
- сам по себе не lazy
- (map f): rf → rf'